

Prueba de Evaluación Continua

Álvaro Jerónimo Sánchez

28/04/2026

ESPECIFICACIONES DE LA PRUEBA

No se han empleado librerías que no sean estándar, y los archivos de cada ejercicio se encuentran en el mismo directorio de este *PDF* con los nombres “*ejercicio_n*” con su extensión correspondiente (*.c*, *.m*). No se han empleados argumentos de línea de comandos ni opciones de compilación, todos los parámetros están dentro del código como variables. No se crearán directorios adicionales a la hora de generar los archivos de datos para evitar confusiones, todos los archivos estarán dentro de este mismo directorio. Este documento ha sido generado con $\text{\LaTeX}$, si fuese necesario, proporcionaré el archivo *.tex* y todas las figuras adjuntas a petición vuestra.

En el directorio se encuentran los siguientes archivos:

```
JERONIMO_SANCHEZ_ALVARO_PEC.zip
├── ejercicio_1.c
├── ejercicio_2.m
├── ejercicio_3.m
└── memoria_resultados.pdf
```

1. Simulación numérica en C

1.1. Discretización del sistema e implementación del algoritmo

Los mecanismos de *reacción-difusión*, propuestos por Alan Turing, explican cómo los patrones presentes en diversos animales son resultado de reacciones entre dos o más sustancias químicas (activador-inhibidor) con pequeñas perturbaciones que acaban causando estructuras periódicas. Este ejercicio es la parte principal de la simulación que generará los datos a partir de los parámetros introducidos. Se resolverán las ecuaciones de dicho modelo creando una red $L \times L$, introduciendo perturbaciones y calculando el *laplaciano* en cada punto discreto. Se ha decidido definir funciones para la mayoría de operaciones necesarias y luego llamarlas en `int main()`.

Se ha inicializado las matrices en el estado homogéneo con los parámetros iniciales dados:

$$L = 100, Du = 1, Dv = 10, a = 0.1, b = 0.9, dt = 0.01s, T = 200s$$

$$U_{0(L \times L)} = \begin{pmatrix} u_0 & \cdots \\ \vdots & \ddots \end{pmatrix}; V_{0(L \times L)} = \begin{pmatrix} v_0 & \cdots \\ \vdots & \ddots \end{pmatrix}$$

siendo: $u_0 = a + b, v_0 = \frac{b}{(a + b)^2}$

Tras inicializar las matrices, se ha añadido un ruido gaussiano aleatorio con una función **gauss** aplicando la transformada de Box-Muller a cada punto de la matriz, cuya fórmula es:

$$z = \sqrt{-2 \ln u_1} \cos(2\pi u_2)$$

Siendo z la perturbación generada, y u_1, u_2 son números aleatorios entre 0 y 1.

Posteriormente, en la función **timestep**, se crean dos nuevas matrices de las mismas dimensiones para ir actualizando con cada paso. Es necesario calcular los valores de v para sustituir en la fórmula de u , aunque la matriz a estudiar en la que se observa el patrón es la u . En cada paso temporal, se calcula el cálculo del tiempo (empleando los datos del paso anterior) en cada punto de la matriz u con las fórmulas:

$$u_{i,j}^{n+1} = u_{i,j}^n + dt \left(D_u \nabla^2 u_{i,j}^n + a - u_{i,j}^n + (u_{i,j}^n)^2 v_{i,j}^n \right)$$

$$v_{i,j}^{n+1} = v_{i,j}^n + dt \left(D_v \nabla^2 v_{i,j}^n + b - (u_{i,j}^n)^2 v_{i,j}^n \right)$$

Y siendo el laplaciano:

$$\nabla^2 u_{i,j} = u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{i,j}$$

Tras recorrer los pasos determinados y escribirlos en un archivo con una función **filesave**, se repite hasta generar los 10 documentos *.dat*, del 0 al 9, correspondientes a 20 000 pasos temporales.

Cabe mencionar que se han aplicado las condiciones de contorno con funciones **if**, igualando los vecinos que “se salen” por un lado (“arriba, abajo, izquierda, derecha”) al primero del lado opuesto. De esta manera no hay comportamientos indeterminados en los bordes.

1.2. Amplitud de la perturbación

Para hallar la amplitud, se calcula el promedio de $(u - u_0)^2$ para cada paso, siendo u el valor de cada punto de la red en dicho paso (que se graba en la misma función `timestep`) y u_0 el valor fijo inicial sin las perturbaciones ($a + b$). Se usa esto para la fórmula:

$$A(t) = \sqrt{\langle (u - u_0)^2 \rangle}$$

El resultado se guarda junto al índice del paso del tiempo con su propia función `ampsave`. Además, se crea un archivo `time.dat` con los índices de cada paso temporal.

2. Análisis de resultados en MATLAB

2.1. Visualización del patrón

En *MATLAB* podremos analizar los resultados obtenidos en la simulación y observar la evolución del patrón generado. Se ha empleado la función `readmatrix` para leer los contenidos de los archivos `.dat` generados con C. Posteriormente, se ha buscado el máximo y mínimo global para usarlos en la escala de color. Se ha usado un bucle que repite los diez pasos de la animación desde principio hasta fin.

El patrón final resultante tiene un aspecto laberíntico, con curvas cuyas tangentes aparentan ser prácticamente paralelas entre ellas en sus caminos, como si girasen alrededor de un mismo radio. Presenta focos circulares con perturbaciones más intensas, y se puede ver cómo las condiciones de contorno se cumplen: si situamos la misma figura duplicada en cualquiera de los bordes, ambas “encajan”. La apariencia de este patrón es la esperada de un patrón de Turing.

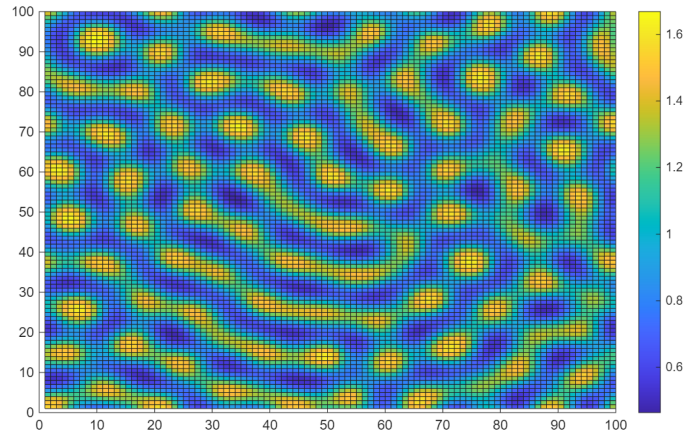


Figura 1: Estado final del patrón generado en la matriz u .

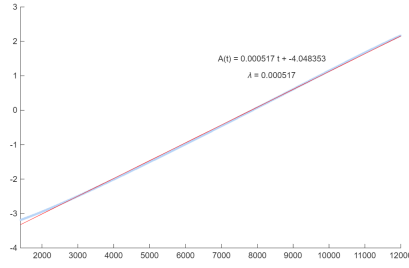


Figura 2: Sección exponencial y ajuste lineal de la amplitud de la perturbación en escala logarítmica con el tiempo.

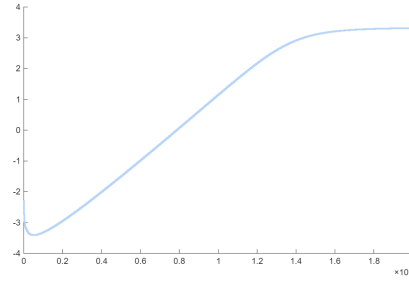


Figura 3: Gráfica completa de la evolución de la amplitud de la perturbación en escala logarítmica con el tiempo.

2.2. Evolución temporal de la perturbación

La evolución de la amplitud de la perturbación se ha representado en una figura separada a la del patrón. Se ha situado este código antes del bucle de la animación para que pueda generarse la gráfica de la amplitud de la perturbación antes del `while` infinito.

Al representar $A(t)$ en escala logarítmica, una recta diagonal representa crecimiento exponencial, por lo que esa será la sección que escojamos. Se ha hecho un ajuste lineal (línea roja), cuya solución se ha imprimido en pantalla, y a la vez se han representado los datos medidos con una gráfica `scatter` de puntos azules.

En la gráfica completa (figura 3), inicialmente, hay una caída vertical hasta que el sistema se estabiliza y empieza a mostrar un crecimiento exponencial. Desde $t = 1.4 \cdot 10^4$ la amplitud decrece hasta ser horizontal, ya que aquí está llegando a su estado final en el que no va a crecer o cambiar más. Esto encaja con lo esperado para el sistema de activador-inhibidor, ya que se observa que a partir de unas pequeñas perturbaciones en un sistema homogéneo, el sistema genera patrones periódicos que crecen hasta estabilizarse.

3. Análisis de estabilidad lineal

3.1. Autovalores del Jacobiano

Se ha inicializado la matriz con los valores iniciales sustituyendo los parámetros $u_0 = a + b$ y $v_0 = \frac{b}{(a+b)^2}$ sabiendo que el Jacobiano es:

$$J = \begin{pmatrix} -1 + 2u_0v_0 & u_0^2 \\ -2u_0v_0 & -u_0^2 \end{pmatrix}$$

Se han hallado sus autovalores con la función `eig()`. Se ha hallado que ambos autovalores tienen parte real negativa `-0.1000`. El tipo de punto fijo es un **foco**, ya que los autovalores son complejos conjugados, y es **asintóticamente estable** ya que ambas partes reales de los autovalores son negativas:

$$\lambda_1 = -0.1000 + 0.9950i; \lambda_2 = -0.1000 - 0.9950i$$

3.2. Barrido en número de onda

Usando el Jacobiano planteado anteriormente, podemos hallar la evolución de las perturbaciones con la matriz:

$$M(k) = J - \begin{pmatrix} D_u k^2 & 0 \\ 0 & D_v k^2 \end{pmatrix}$$

Se quiere hallar el valor de k para el cual $M(k)$ es máximo para un rango $[0, 2]$. Se ha decidido analizar 600 puntos de dicho intervalo. Se ha representado la parte real del autovalor dominante en función de k (relación de dispersión), y se ha elegido el valor de k máximo para el que la función sea máxima. Se ha marcado el punto en la gráfica y se ha mostrado su valor en pantalla.

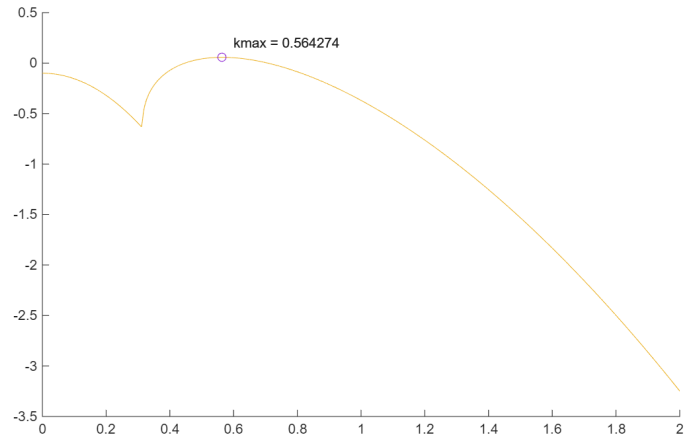


Figura 4: Relación de dispersión: parte real del autovalor dominante en función de k . Se ha marcado k_{mx} con un círculo magenta.

Nota: Se ha buscado la k_{mx} empleando la función `find()` de MATLAB, ya que en el guión parece especificarse que hay que hallar el valor máximo de ésta. Realmente sólo sería necesario en el caso de que haya dos valores de k con un mismo máximo, en cuyo caso se elegiría la k mayor entre las dos.